

WEEK 6 PRACTICAL

QUESTION 1

Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

Aim

To implement client-server communication using Named Pipes (FIFOs), where clients send messages to a server and the server processes and responds to them.

Program — Server (server.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>

#define FIFO1 "/tmp/client_to_server"
#define FIFO2 "/tmp/server_to_client"

int main() {
    char message[100];
    char response[150];

    mkfifo(FIFO1, 0666);
    mkfifo(FIFO2, 0666);

    printf("Server started. Waiting for client...\n");

    int fd1 = open(FIFO1, O_RDONLY);
    int fd2 = open(FIFO2, O_WRONLY);

    while (1) {
        memset(message, 0, sizeof(message));

        int n = read(fd1, message, sizeof(message) - 1);

        if (n <= 0)
            break;

        message[n] = '\0';

        printf("Client message: %s\n", message);
```

```

        if (strcmp(message, "exit") == 0) {
            strcpy(response, "Server shutting down.");
            write(fd2, response, strlen(response) + 1);
            break;
        }

        snprintf(response, sizeof(response),
                 "Server processed: %s", message);

        write(fd2, response, strlen(response) + 1);
    }

    close(fd1);
    close(fd2);

    unlink(FIFO1);
    unlink(FIFO2);

    return 0;
}

```

Program — Client (client.c)

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

#define FIFO1 "/tmp/client_to_server"
#define FIFO2 "/tmp/server_to_client"

int main() {
    char message[100];
    char response[150];

    int fd1 = open(FIFO1, O_WRONLY);
    int fd2 = open(FIFO2, O_RDONLY);

    printf("Connected to server.\n");

    while (1) {
        printf("Enter message: ");
        fgets(message, sizeof(message), stdin);

        message[strcspn(message, "\n")] = '\0';

        write(fd1, message, strlen(message) + 1);

        int n = read(fd2, response, sizeof(response) - 1);

        if (n <= 0)
            break;

        response[n] = '\0';

        printf("Server response: %s\n", response);
    }
}

```

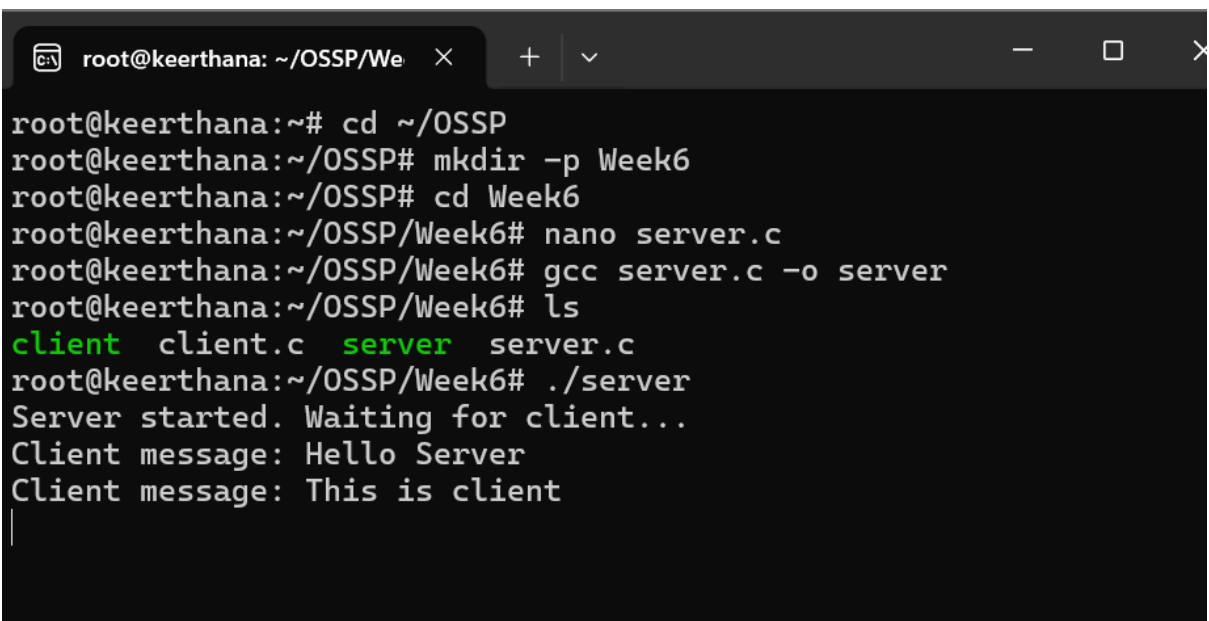
```
        if (strcmp(message, "exit") == 0)
            break;
    }

    close(fd1);
    close(fd2);

    return 0;
}
```

Output

Terminal 1 — Server

A terminal window titled "root@keerthana: ~/OSSP/We" with standard window controls. The terminal shows the following commands and output:

```
root@keerthana:~# cd ~/OSSP
root@keerthana:~/OSSP# mkdir -p Week6
root@keerthana:~/OSSP# cd Week6
root@keerthana:~/OSSP/Week6# nano server.c
root@keerthana:~/OSSP/Week6# gcc server.c -o server
root@keerthana:~/OSSP/Week6# ls
client client.c server server.c
root@keerthana:~/OSSP/Week6# ./server
Server started. Waiting for client...
Client message: Hello Server
Client message: This is client
```

Terminal 2 — Client

```
root@keerthana: ~/OSSP/Week6
root@keerthana:~/OSSP/Week6# nano client.c
root@keerthana:~/OSSP/Week6# gcc client.c -o client
root@keerthana:~/OSSP/Week6# ./client
Connected to server.
Enter message: Hello Server
Server response: Server processed: Hello Server
Enter message: This is client
Server response: Server processed: This is client
Enter message: exit
Server response: Server shutting down.
root@keerthana:~/OSSP/Week6#
```

Observation

The named FIFOs provide communication between processes without requiring them to be related by `fork()`. The client writes messages to the client-to-server FIFO, while the server reads and processes them and sends responses through the server-to-client FIFO.

When multiple clients communicate simultaneously, messages are placed into the FIFO according to the order in which writes occur. A single FIFO does not automatically provide separate communication channels for individual clients, so a practical multi-client server would normally use separate response FIFOs or another mechanism to identify each client.

Result

The client-server communication system was successfully implemented using Named Pipes (FIFOs), and the behavior of FIFO communication with multiple clients was analyzed.

QUESTION 2

Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

Aim

To implement POSIX signal handling using SIGINT, SIGTERM, and SIGUSR1, and demonstrate how signal handlers respond to asynchronous events.

Program

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

void signal_handler(int signum) {
    if (signum == SIGINT) {
        printf("\nSIGINT received. Interrupt signal handled.\n");
    }
    else if (signum == SIGTERM) {
        printf("\nSIGTERM received. Termination signal handled.\n");
    }
    else if (signum == SIGUSR1) {
        printf("\nSIGUSR1 received. User-defined signal handled.\n");
    }
}

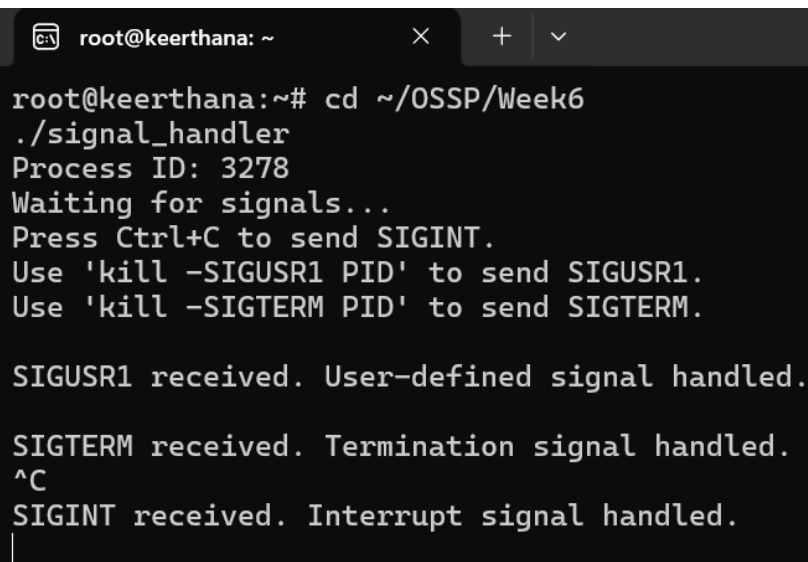
int main() {
    signal(SIGINT, signal_handler);
    signal(SIGTERM, signal_handler);
    signal(SIGUSR1, signal_handler);

    printf("Process ID: %d\n", getpid());
    printf("Waiting for signals...\n");
    printf("Press Ctrl+C to send SIGINT.\n");
    printf("Use 'kill -SIGUSR1 PID' to send SIGUSR1.\n");
    printf("Use 'kill -SIGTERM PID' to send SIGTERM.\n");

    while (1) {
        pause();
    }

    return 0;
}
```

Output

A terminal window titled 'root@keerthana: ~' with standard window controls. The user has navigated to the directory ~/OSSP/Week6 and executed the program ./signal_handler. The program outputs the process ID 3278 and enters a loop waiting for signals. It displays instructions on how to send SIGINT (Ctrl+C), SIGUSR1 (kill -SIGUSR1 PID), and SIGTERM (kill -SIGTERM PID). The output shows three signals being received and handled: SIGUSR1, SIGTERM, and SIGINT. The cursor is positioned at the end of the last line of output.

```
root@keerthana:~# cd ~/OSSP/Week6
./signal_handler
Process ID: 3278
Waiting for signals...
Press Ctrl+C to send SIGINT.
Use 'kill -SIGUSR1 PID' to send SIGUSR1.
Use 'kill -SIGTERM PID' to send SIGTERM.

SIGUSR1 received. User-defined signal handled.

SIGTERM received. Termination signal handled.
^C
SIGINT received. Interrupt signal handled.
|
```

Observation

The program registers a signal handler for SIGINT, SIGTERM, and SIGUSR1. When any of these signals is delivered, the operating system asynchronously invokes the corresponding handler.

- **SIGINT** — normally generated using Ctrl+C.
- **SIGTERM** — requests termination of a process.
- **SIGUSR1** — a user-defined signal that can be used for application-specific events.

The pause() system call keeps the process waiting until a signal is received.

Result

The POSIX signal handling program was successfully implemented. The program captured SIGINT, SIGTERM, and SIGUSR1 and demonstrated asynchronous event handling using signal handlers.